Dependency-Preserving Evidence Packing for Numerical Question Answering over Financial Reports

Pushpendra Singh
Roll No. 240150026
IIT Guwahati

Badri Bishal Das
Roll No. 240150006
IIT Guwahati

DA331 Big Data Analytics: Tools and Techniques, course project report

Abstract

The evidence given to a language model that answers questions about a financial report is limited by the size of its context window. Since most answers are found in tables, and since a table cell cannot be interpreted without its row and column labels, the way in which labels are written into the prompt determines how much evidence can be included. Writing the labels beside every number costs between 2.2 and 8.3 times as many tokens per cell as writing the number alone. In this paper we represent each report as a dependency graph that links every number to the labels it requires, and we pack evidence so that each label is written only once. A PySpark pipeline was used to build and verify the graph for six datasets, comprising 1.36 million facts and 1.88 million label links. Over the 9,268 development questions, the proposed method retained the gold evidence more often than label-repeating packing on all six datasets, by 0.3 to 4.0 points at the chosen budgets. This advantage decreases as the budget grows and is reversed on two datasets at a budget of 200 tokens. In a paired experiment with Gemini 3.5 Flash-Lite, answer scores were at least as high on all six datasets, and the gain of 11.3 points on TAT-QA was the only one whose confidence interval excluded zero.

1 Introduction

Analysts frequently consult annual reports for specific quantities such as a margin, a growth rate or the ratio of two line items. Answering such questions automatically is an instance of question answering over mixed table and text data. A large language model (LLM) can perform this task, but it can only use the text in its prompt, and a report is usually much longer than the evidence budget available to the system. A few facts must therefore be retrieved and then *packed* into that budget.

Tables make this step difficult. Figure 1 shows a table taken from FinQA, for which the question

company	paym. vol.	total vol.	transactions	cards
mastercard	1697	2276	27.0	916
amer. express	637	647	5.0	86
discover	102	119	1.6	57

"What is the average payment volume per transaction for American Express?"

$$637 \div 5.0 = 127.4.$$

Figure 1: Running example from the FinQA development set (V/2008/page_17, rows abridged). The two top-ranked cells differ only by their column header.

asks for the average payment volume per transaction of American Express. A retriever ranks the cells 637 and 647 highest, and it is only their column headers which indicate that the first is the payments volume and the second the total volume. If the two numbers are packed without labels they cannot be told apart. If they are packed with all of their labels, as in the template "the revenue of 2019 is 100" (Chen et al., 2021; Arun et al., 2023), each cell costs several times as many tokens and fewer cells can be included. What is needed is a packing method that retains the labels of every number and is still economical with the budget.

We make four contributions. First, we describe dependency-preserving packing, in which a number is admitted together with the labels it points to and a label that has already been packed costs nothing, and we derive the token trade-off of this method: a fixed cost for structure, after which cells are 2.2 to 8.3 times cheaper. Second, we present a PySpark data layer which normalizes six datasets into a single verified store of facts and label links. Third, we report a coverage audit over all 9,268 development questions, five budgets and three packing methods, for which no model is required. Finally, we report a paired answer experiment with Gemini 3.5 Flash-Lite in which the model copies numbers from the evidence and code checks them and carries out the computation.

The remainder of this paper is organized as follows. Section 2 reviews related work, Sections 3 and 4 state the problem and the data, and Section 5 gives the method. Section 6 covers the PySpark implementation and Sections 7 and 8 the experiments, and Sections 9 and 10 close the paper.

2 Related Work

Numerical reasoning over earnings reports was introduced with FinQA (Chen et al., 2021), in which answers are expressed as programs. TAT-QA (Zhu et al., 2021) mixes table and paragraph evidence and has several answer types, while ConvFinQA (Chen et al., 2022) spreads the reasoning over the turns of a conversation. Learning to Imagine (Li et al., 2022) adds hypothetical questions on TAT-QA documents, and MultiHiertt (Zhao et al., 2022) uses hierarchical tables with long passages of text. HybridQA (Chen et al., 2020) links Wikipedia tables to passages and contains little arithmetic, and for this reason we treat it as a test of whether the method transfers outside finance.

Several components of our pipeline are well established. Program of Thoughts (Chen et al., 2023) delegates calculation to an interpreter, and TAT-LLM (Zhu et al., 2024) divides the task into extraction, reasoning and execution; our executor follows this line of work. Arun et al. (2023) report that naive serialization handled multi-index and non-standard tables poorly, and they used an LLM to write the serialized text instead. The systems closest to ours are FINDER (Khatuya et al., 2025), which combines generative retrieval with in-context examples chosen per question, and FinCARDS (Zhou et al., 2026), which uses finance-aware schema alignment and tournament reranking to select evidence with an audit trail.

These systems differ from ours in that they change what is retrieved or how the model reasons over it. In the present work the model, the ranking and the budget are held fixed and a single decision is varied, namely how a label is paid for. We regard the controlled measurement of this decision across six benchmarks as our contribution, and we make no claim of novelty for structured evidence or executable reasoning. Since our literature search was targeted, we cannot exclude that this exact combination has been tried elsewhere.

Dataset	Questions	Challenge	D	O	Budget
FinQA	883	programs	1,041	1,112	800
ConvFinQA	1,490 turns	history	979	1,062	800
TAT-QA	1,668	spans, scale	396	592	400
TAT-HQA	824	hypotheticals	396	592	400
HybridQA	3,466	passage links	8,928	11,142	1,500
MultiHiertt	1,044	hierarchy	3,693	6,435	1,500

Sources (GitHub): czyssrs/FinQA, czyssrs/ConvFinQA, NExTplusplus/TAT-QA, NExTplusplus/TAT-HQA, wenhuchen/HybridQA, psunlpgroup/MultiHiertt.

Table 1: Development data. D and O: median document size in tokens with each label written once and with labels repeated in every cell.

3 Problem Statement and Objectives

The problem assigned to us is numerical reasoning over financial reports on FinQA, TAT-QA, ConvFinQA and HybridQA. The question we investigate is the following: *with the same model and the same evidence budget, does keeping each number's headers, units, hierarchy and links improve answers over packing that repeats labels inline?* The hypothesis itself predicts two limits. Because labels consume budget, the method may lose when almost nothing fits, and when the whole document fits there is nothing to choose between methods.

Our objectives were (O1) a single pipeline over the four assigned datasets together with TAT-HQA and MultiHiertt, scored by the official evaluators; (O2) a comparison of dependency-preserving packing against label-repeating and length-matched baselines at a fixed budget; (O3) a verified PySpark preparation of all six datasets; (O4) a paired answer test; and (O5) answers that are grounded in the evidence.

4 Datasets

We use the development splits of the six datasets listed in Table 1, each obtained from its official repository at a pinned commit, along with the supplied document context and the official scorers. FinQA and ConvFinQA share documents, as do TAT-QA and TAT-HQA, and results are therefore never pooled. HybridQA is drawn from Wikipedia and is included to test transfer. As a check on the scorers, the gold answers of 200 development questions were passed through each one. This gave 100% on five datasets and 90.5% on MultiHiertt, whose span path drops minus signs.



Figure 2: Pipeline. The highlighted step is the only one that varies.

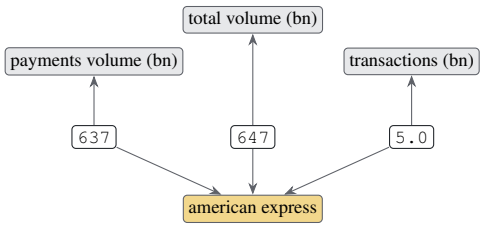


Figure 3: Dependency graph for three cells of the running example. The row label is a single shared node.

5 Proposed Method

The pipeline is shown in Figure 2. Only the packing step differs between the three methods that we compare.

5.1 Evidence as a Dependency Graph

Each document is converted by a reader into *facts*, where a fact consists of raw text, its provenance and the IDs of the facts needed to read it (Figure 3). A table cell depends on its column header, row label, corner header and section row. TAT-QA cells depend in addition on unit paragraphs such as "in thousands", a HybridQA passage depends on the cell that links to it, and MultiHiertt cells depend on multi-row HTML headers and section headings. Headers are inferred from the layout and are flagged as uncertain. Gold answers do not enter the prompts, and none of the 443 prompts that we inspected contains a gold label field.

5.2 Packing under the Budget

BM25 is used to rank cells and paragraphs against the question and any earlier questions. Candidates are admitted in rank order as whole bundles. Strings are never cut, and a bundle that does not fit is skipped so that smaller ones may still enter. Three methods spend the same budget. In **Ordinary** packing (O), each fact carries its structural labels inline, so that the baseline is not deprived of labels. In **Dependency** packing (D), each fact enters with the recursive closure of its headers, units, hierarchy and link anchors, and labels already packed are free. In **Length-matched** packing (L), ordinary facts are packed with their document neighbors up to the token count reached by D, which tests whether more text alone is of benefit. Tokens are counted locally on the evidence text as UTF-8 bytes divided by four.

The ranking is deterministic. For a set of query terms q and a candidate f, whose text includes its direct dependencies,

$$\mathrm{score}(f) = \sum_{t \in q} \mathrm{idf}(t)\, \frac{\mathrm{tf}_{t,f}\,(k_1+1)}{\mathrm{tf}_{t,f} + k_1\big(1 - b + b\,\frac{|f|}{\overline{|f|}}\big)}, \tag{1}$$

with $k_1 = 1.2$, $b = 0.75$ and $\mathrm{idf}(t) = \log\big(1 + \frac{N - n_t + 0.5}{n_t + 0.5}\big)$ computed within the one document. Ties are broken on the fact ID. In ConvFinQA the earlier turns enter first, newest first, for every method, so that conversation state costs all three the same amount. After selection, facts are presented in document order.

The packing loop is given in Figure 4. Let $c(x) = \lceil \mathrm{bytes}(x)/4 \rceil$ and let m_ℓ be the number of selected facts that use label ℓ. For a selected set S with label set $L(S)$, the two costs are approximately

$$C_O(S) \approx \sum_{f \in S} c(f) + \sum_{\ell \in L(S)} m_\ell\, c(\ell), \tag{2}$$

$$C_D(S) \approx \sum_{f \in S} c(f) + \sum_{\ell \in L(S)} c(\ell) + \sum_{p \in P(S)} c(p), \tag{3}$$

where separators and rounding have been ignored. The set $P(S)$ contains those items which D pays for and O never carries. The baseline inlines only structural labels, and a unit paragraph is therefore charged to D alone; copying such a paragraph into every cell would have penalized the baseline unfairly. Subtracting the two expressions gives

$$C_O - C_D \approx \sum_{\ell \in L(S)} (m_\ell - 1)\, c(\ell) - \sum_{p \in P(S)} c(p). \tag{4}$$

It can be seen from Equation 4 that D saves tokens only through labels shared by at least two selected facts. A single selected cell has $m_\ell = 1$ for all of its labels and saves nothing, and at a very small budget few cells can enter at all. D may therefore lose at low budgets, and it should gain once several cells of one table have been packed.

Figure 5 illustrates this trade-off on the running example. The first cell costs 18 tokens under D as against 15 under O, because its headers come with it, whereas four cells cost 50 as against 57.

Input: ranked candidates R, budget B, method
$used \leftarrow 0$; $T \leftarrow \emptyset$
for $f \in R$ **do**
  if method is D **then** $b \leftarrow \text{closure}(f) \cup \{f\}$
  else $b \leftarrow \{\text{inline_labels}(f)\}$
  $n \leftarrow b \setminus T$ *// packed facts are free*
  if $used + \sum_{x \in n} c(x) \leq B$ **then**
    $T \leftarrow T \cup n$; $used \leftarrow used + \sum_{x \in n} c(x)$
  else skip f and continue
return T in document order

Figure 4: Packing loop shared by O and D (`core.py`). L runs the O branch over each candidate and its two document neighbors, with D's final token count as its budget.

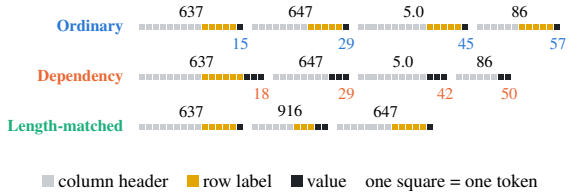


Figure 5: Packing the American Express row. Numbers below each group are cumulative tokens.

L spends its tokens on a neighboring Mastercard cell and loses 5.0. Over all development tables (Table 2), a labeled cell costs 15.1 tokens in FinQA as against 3.6 for a bare one (4.2×), 15.6 as against 7.0 in HybridQA, and 24.7 as against 3.0 in MultiHiertt (8.3×). The median TAT-QA document requires 396 tokens with labels written once and 592 with labels repeated, and its budget is 400 (Table 1).

5.3 Grounded Answers

An earlier version of the system asked the model to cite each operand by evidence ID and character offsets. That version was recorded as reaching 2 of 883 FinQA development questions with a local quantized model, with about 90% of the outputs failing validation. The raw outputs of that run are not available to us and the model has since been changed as well, so the figures are given only as recorded and the later improvement cannot be credited to the output format alone.

In the final design the model returns a JSON plan of official operations whose arguments are numbers copied from the evidence, e.g. `divide("637","5.0")`. Code then locates each number in the packed evidence or in the question, rejects the plan if a number is missing, and computes the result in a bounded decimal interpreter. This prevents the use of invented numbers but does not prevent the choice of a wrong number that is present: `divide("647","5.0")` passes and returns 129.4, and only the labels allow the model to avoid this choice.

Dataset	Docs	Cells	Labeled	Bare	Ratio
FinQA	883	12,771	15.1	3.6	4.2
ConvFinQA	421	6,336	15.3	3.7	4.1
TAT-QA	278	6,402	15.3	3.2	4.7
TAT-HQA	274	6,324	15.3	3.2	4.7
HybridQA	3,053	170,179	15.6	7.0	2.2
MultiHiertt	1,044	177,108	24.7	3.0	8.3

Table 2: Mean tokens per table cell with labels inline and bare, over all development tables. Ratio is computed before rounding.

The prompt permits five kinds of answer, namely extract, multi-extract, count, arithmetic and abstain. Arithmetic plans use add, subtract, multiply, divide, exp and greater, and an argument may be a copied number, a reference to an earlier step such as `#0`, or a named constant. The model also states a scale, so that a fraction of 0.25 can be reported as 25%. For a hypothetical question it uses the changed value given in the question. Each question receives one call and there is no repair loop, so that a rejected plan scores zero.

6 Big Data Implementation with PySpark

The six datasets are distributed as JSON (FinQA, TAT-QA, TAT-HQA), a zip archive (ConvFinQA), HTML tables (MultiHiertt) and a linked corpus of 15,314 Wikipedia tables (HybridQA). Python readers normalize these into three JSON-lines tables: documents, keyed by a SHA-256 content hash so that repeated contexts are stored once; questions, without labels; and facts, each of which holds its dependency IDs as an array. Spark 4.3.0 then runs four Spark SQL jobs over these DataFrames (Figure 6). The first joins questions to documents. The second applies `explode` to the dependency array of each fact to obtain an edge table, which constitutes the links of the graph. The third performs a `left_anti` join of the edges against the fact IDs, the result of which must be empty. The fourth is a self-join of documents on content hash, which finds identical documents across datasets.

The store is written as Parquet partitioned by dataset and split, so that one dataset can be read without scanning the others. The job runs in local mode with 8 shuffle partitions and a 3 GB driver, and it fails unless its join count and hash, edge

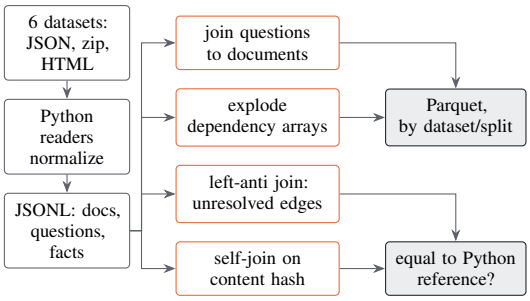


Figure 6: PySpark data layer. Spark SQL jobs in orange.

count and overlap list equal those of an independent single-process Python implementation.

The resulting store holds 1,360,644 facts from 9,375 questions over 4,395 deduplicated documents, with 1,884,748 edges of which 0 are unresolved. The self-join found 491 identical document pairs across datasets, which gives a measured justification for not pooling results. Parquet occupies 0.18 GB as against 0.95 GB of JSON lines, a reduction by a factor of 5.1.

Table 3 gives the time taken by the same jobs at three data sizes and on 1, 2, 4 and 8 cores of an eight-core laptop. On the full data Spark is fastest at four cores, at 37.0 s (36,700 facts/s), and slowest at eight, at 65.6 s. Single-process Python takes 15.8 s (86,000 facts/s). With 1.4 million rows and a peak of 1.7 GB the data fits in memory, and JVM start-up, shuffles and the collection of results to the driver for the equality check are the likely overheads. Parsing the raw files into facts takes 120 s in Python and is the real bottleneck. At this scale, then, the benefit of Spark is a verified and partitioned columnar store, and we draw no conclusion about its scaling on a cluster.

The coverage audit runs 139,020 packings (9,268 questions $\times$ 5 budgets $\times$ 3 methods) over eight worker processes in 679 s. Model calls were subject to a free-tier quota of 500 requests per day for Gemini 3.5 Flash-Lite, and runs were interrupted by it. For this reason a SQLite ledger stores every model response under a frozen run contract (question IDs, seed, input and code hashes), which allows a run to resume after a quota stop and allows re-scoring without new calls.

7 Experimental Setup

A budget of 4,000 tokens proved unusable, since four of the datasets were never truncated at that size in a 40-question sample and the methods could not differ. Each budget was then set from the document sizes, without reference to the labels: 800 for FinQA and ConvFinQA and 400 for TAT-QA and TAT-HQA (the largest tested values truncating about 70% of sampled documents), and 1,500 for HybridQA and MultiHiertt, which remain 93% truncated even at 4,000.

Size	Facts	Python	1	2	4	8	Peak GB
100	74,655	1.1	27.8	28.4	23.5	23.2	0.9
1000	507,594	6.0	23.4	21.2	20.2	33.4	1.4
full	1,360,644	15.8	54.5	38.4	37.0	65.6	1.7

Table 3: Wall time (s): single-process Python and Spark on 1–8 cores. Size is records per source dataset. All runs equal the Python reference.

Three measurements were made. The *coverage audit* checks, without a model, whether every gold operand or answer phrase survives packing; for the running example the gold program is `divide(637, const_5)`, so that only 637 must survive. *Block A* answers 50 questions per dataset with each of O, D and L (45 turns in 15 ConvFinQA conversations). *Block B* compares the executor with direct answering on D evidence for 40 questions per dataset.

The configuration of the models is listed in Table 4. Both are used as served, with no training or fine-tuning, and each question is one API call. The two blocks are never compared with each other. Support for OpenAI models exists in the code but was not used, as the available account had no credits.

Scoring uses official execution accuracy for FinQA and ConvFinQA and official F1 elsewhere. Block B uses a 1%-tolerance numeric measure for FinQA and ConvFinQA, since direct answers have no program. Abstained and rejected answers score zero. Intervals are obtained by paired bootstrap resampling of whole documents (2,000 resamples, seed 2626) and are uncorrected across the six comparisons in each block.

8 Results

8.1 Evidence Coverage

As can be seen from Table 5 and Figure 7, D retains more gold evidence than O on all six datasets at the chosen budgets. Questions retained by only one method favor D by 146 to 7 on HybridQA and 42 to 3 on MultiHiertt, while FinQA (9 to 1) is weaker and ConvFinQA (6 to 1) is not significant. The difference is largest where documents

	Block A (packing)	Block B (executor)
Model	Gemini 3.5 Flash-Lite	Gemini 3.1 Flash-Lite
Served version	3.5-flash-lite-07-2026	3.1-flash-lite-05-2026
Training	none (as served)	none (as served)
Output	schema-bound JSON	schema-bound JSON
Max output tokens	2,048	2,048
Thinking level	low	low
Temperature	provider default	provider default
Responses	885	472
Median tokens in / out	1,403 / 129	1,214 / 109
Sample (seed 2626)	50 per dataset	40 per dataset

Table 4: Model and inference configuration (from the frozen run contracts).

Dataset	n	Cap	O	D	L	D+/O+	p
FinQA	845	800	96.3	97.3	92.2	9/1	0.021
ConvFinQA	1486	800	97.1	97.4	92.7	6/1	0.125
TAT-QA	1636	400	90.1	91.7	82.6	41/15	<.001
HybridQA	3466	1500	72.0	76.0	67.7	146/7	<.001
TAT-HQA	792	400	84.2	86.9	75.4	33/12	0.002
MultiHiertt	1043	1500	84.9	88.6	75.3	42/3	<.001

Table 5: Questions with all gold evidence retained (%). D+/O+: kept only by D or only by O; p: two-sided sign test.

most exceed the budget and smallest on FinQA and ConvFinQA, where labels add only 7–8% to a document. L is below O at every budget, which suggests that additional neighboring text is no substitute for labels.

The dependence on budget is shown in Table 6. The difference shrinks as the budget grows and is zero at 4,000 tokens on the four smaller datasets. At 200 tokens the fixed cost of structure is not recovered, and D loses 1.8 points on ConvFinQA (36 against 63 discordant) and 0.8 on MultiHiertt (50 against 58), although both recover by 400. ConvFinQA spends budget on history first and MultiHiertt has the most expensive labels, and these seem plausible explanations. HybridQA is the only dataset on which D leads at every budget, and it retains a difference at 4,000 together with MultiHiertt because their documents still do not fit. This behavior is consistent with Equation 4.

8.2 Effect on Answers

Table 8 gives the answer scores obtained with Gemini 3.5 Flash-Lite. D scores at least as high as O on all six datasets, by 0 to 11.3 points, but the interval excludes zero only for TAT-QA, at +11.3 [+4.1, +19.9]. For MultiHiertt (+9.8) and FinQA (+2.0) the lower bound is at zero. The pattern follows that of coverage, with the largest gains on hierarchical tables and none on ConvFinQA, whose documents nearly fit.

Dataset	200	400	800	1,500	4,000
FinQA	+0.8	+0.5	+0.9	+0.1	0.0
ConvFinQA	−1.8	+0.9	+0.3	0.0	0.0
TAT-QA	+3.2	+1.6	+0.7	+0.3	0.0
TAT-HQA	+3.5	+2.7	+0.5	+0.5	0.0
HybridQA	+6.8	+5.8	+5.0	+4.0	+2.5
MultiHiertt	−0.8	+4.0	+4.8	+3.7	+1.5

Table 6: Coverage gap D−O in percentage points at each budget, computed before rounding, so a gap can differ by 0.1 from Table 5.

Dataset	O	D	L
FinQA	4 / 3	3 / 2	5 / 1
ConvFinQA	0 / 0	0 / 1	0 / 1
TAT-QA	7 / 1	1 / 1	6 / 2
TAT-HQA	3 / 2	3 / 4	2 / 6
HybridQA	1 / 2	0 / 3	1 / 3
MultiHiertt	7 / 3	5 / 3	12 / 1
Total (of 295)	22 / 11	12 / 14	26 / 14

Table 7: Block A outcomes that score zero: abstained / rejected, per packing method.

Table 7 shows how the methods fail. Across the 295 paired questions the model abstains 22 times with O, 12 times with D and 26 times with L. Most of the reduction comes from TAT-QA, where abstentions fall from 7 to 1, and this is the same dataset on which the score improves. An abstention means that the model has judged the evidence to be insufficient, as would be expected when a required cell has not fit. The numbers of rejected plans, 11 for O and 14 for D, are too small to separate the methods. L has as many tokens as D without the labels, and yet it abstains most often.

Figure 8 replays one TAT-QA question from the saved model outputs, which asks for the ratio of inventory reserves to accrued compensation in 2018 at a budget of 400. Both methods pack the same two paragraphs (355 tokens). O fills the remainder with three labeled cells and stops at 398, whereas the accrued-compensation cell, 358, would need 17 more tokens; the model reports that accrued compensation is missing from the evidence and abstains. D pays 33 tokens for the labels once and 3 per cell, includes 358, and the plan `divide("948","358")` returns 2.65, the gold answer.

8.3 Executor against Direct Answering

The results with Gemini 3.1 Flash-Lite in Table 9 show no consistent gain from the executor. Direct answering leads on HybridQA by 16.4 points with

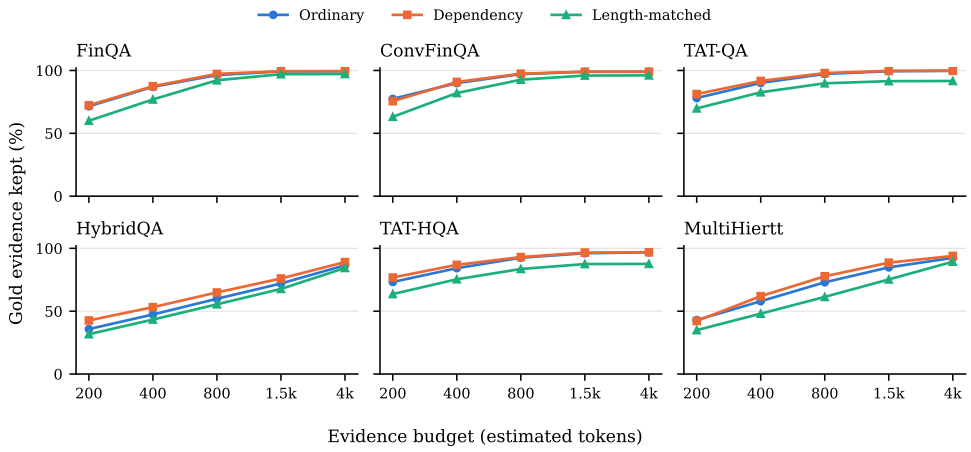


Figure 7: Gold evidence retained against budget, full development splits.

Dataset	n	O	D	L	D−O	95% CI
FinQA	50	62.0	64.0	66.0	+2.0	[+0.0, +6.2]
ConvFinQA	45	73.3	73.3	68.9	+0.0	[−6.0, +6.4]
TAT-QA	50	60.2	71.5	69.4	+11.3	[+4.1, +19.9]
HybridQA	50	54.9	60.1	49.8	+5.3	[−2.4, +13.0]
TAT-HQA	50	56.7	60.7	52.7	+4.0	[−6.4, +14.5]
MultiHiertt	50	47.0	56.8	41.0	+9.8	[+0.0, +20.0]

Table 8: Block A answer scores (%), Gemini 3.5 Flash-Lite. Execution accuracy for FinQA and ConvFinQA, F1 otherwise.

Dataset	n	Executor	Direct	Diff.	95% CI
FinQA	40	62.5	67.5	−5.0	[−17.5, +7.0]
ConvFinQA	36	75.0	75.0	+0.0	[−8.6, +7.3]
TAT-QA	40	65.2	58.7	+6.5	[−7.6, +20.7]
HybridQA	40	50.2	66.6	−16.4	[−30.6, −2.0]
TAT-HQA	40	60.0	56.7	+3.3	[−12.0, +19.3]
MultiHiertt	40	63.7	41.7	+22.1	[+2.5, +40.0]

Table 9: Block B (%), Gemini 3.1 Flash-Lite: executor against direct answering on D evidence. MultiHiertt is official F1; under one numeric rule the gap is +6.2.

an interval excluding zero, which is unsurprising since its answers are short text spans with no arithmetic. The lead of 22.1 points for the executor on MultiHiertt is an artifact of scoring: direct answers write percentages (-29.92 for a gold -0.2992), which the official span path marks as wrong. Under one numeric rule for both sides the difference is +6.2 [−6.2, +20.0], and we therefore draw no conclusion from it. The main value of the executor is that every answer can be traced to specific evidence.

8.4 Error Analysis

Table 10 classifies the 102 of 236 Block B executor answers that did not receive full credit under the primary measure. Plans rejected because a number

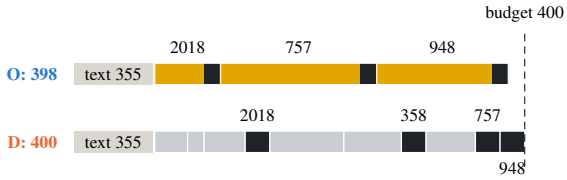


Figure 8: TAT-QA question at budget 400. Yellow: labels inline; dark: values; grey: labels written once. O (398 tokens) has no room for 358; D (400) fits it.

Dataset	Val	Sign	Span	Grnd	Abst	Other	All
FinQA	7	3	0	2	4	1	17
ConvFinQA	6	2	0	1	0	0	9
TAT-QA	2	2	13	0	3	0	20
HybridQA	0	0	19	2	4	0	25
TAT-HQA	9	1	2	3	1	0	16
MultiHiertt	6	2	2	0	3	2	15

Table 10: Block B executor answers without full credit: wrong value (Val), wrong sign, wrong span or text (Span), value not in evidence (Grnd), abstained (Abst), other.

was not in the evidence account for only 8 of these. Wrong values (30) and wrong spans (36) make up most of the remainder and divide by dataset much as one would expect. FinQA, ConvFinQA, TAT-HQA and MultiHiertt fail mostly on values, such as a percentage change computed in the reversed direction, while TAT-QA and HybridQA fail on spans that are wrong or too long. A further 10 answers have the correct magnitude and the wrong sign.

9 Reproducibility

Each run has a frozen contract holding the question IDs, seed, source and input hashes, prompt, schema, settings and code hashes. Official eval-

uators are fetched at pinned commits, the main result tables are written from saved outputs by `scripts/report_tables.py`, and the test suite makes no API calls. Python 3.12 and, for Spark, Java 17 are required. The principal commands are:

```
scripts/fetch_upstream.sh
scripts/run_spark.sh 100 1000 full
python scripts/coverage_audit.py
scripts/run_v11.sh a \
  gemini-3.5-flash-lite eval finqa \
  tatqa tathqa convfinqa hybridqa \
  multihiertt
python scripts/report_tables.py
```

10 Limitations

Coverage measures the presence of evidence and says nothing about its correct use. Free-tier quotas restricted the samples, and with 50 questions most per-dataset answer gains remain within noise; a block interrupted by a quota stop was resumed under the final code, with saved answers re-parsed. Headers are inferred and may be wrong, budgets are estimated tokens, and BM25 misses paraphrases. Hosted models can change between runs. The overheads of Spark were not profiled, the explanations offered for the losses at 200 tokens were not tested, m_ℓ was not measured directly, and the error classification does not show whether the required cell had been packed. Finally, the test labels of ConvFinQA and HybridQA are not public, and all results are accordingly on development splits.

11 Conclusion and Future Work

Writing each label once and linking every number to it retained more of the evidence needed for a question at the same budget on all six datasets at the chosen budgets, particularly where documents exceed the budget, with a measured loss at 200 tokens on two datasets. The answers of Gemini 3.5 Flash-Lite follow the same direction, with a significant gain on TAT-QA, and the 1.36-million-fact graph on which the method runs is built and verified with PySpark. In future work we intend to use larger answer samples, to make the closure budget-aware so that a header is paid for only when several packed facts share it, and to improve header inference for nested tables.

References

Abhinav Arun, Ashish Dhiman, Mehul Soni, and Yibei Hu. 2023. Numerical reasoning for financial reports. *arXiv preprint arXiv:2312.14870*.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. 2023. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*.

Wenhu Chen, Hanwen Zha, Zhiyu Chen, Wenhan Xiong, Hong Wang, and William Yang Wang. 2020. HybridQA: A dataset of multi-hop question answering over tabular and textual data. In *Findings of EMNLP*.

Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, and William Yang Wang. 2021. FinQA: A dataset of numerical reasoning over financial data. In *Proceedings of EMNLP*.

Zhiyu Chen, Shiyang Li, Charese Smiley, Zhiqiang Ma, Sameena Shah, and William Yang Wang. 2022. ConvFinQA: Exploring the chain of numerical reasoning in conversational finance question answering. In *Proceedings of EMNLP*.

Subhendu Khatuya, Shashwat Naidu, Pawan Goyal, and Niloy Ganguly. 2025. Program of thoughts for financial reasoning: Leveraging dynamic in-context examples and generative retrieval. In *Proceedings of EMNLP*.

Moxin Li, Fuli Feng, Hanwang Zhang, Xiangnan He, Fengbin Zhu, and Tat-Seng Chua. 2022. Learning to imagine: Integrating counterfactual thinking in neural discrete reasoning. In *Proceedings of ACL*.

Yilun Zhao, Yunxiang Li, Chenying Li, and Rui Zhang. 2022. MultiHiertt: Numerical reasoning over multi hierarchical tabular and textual data. In *Proceedings of ACL*.

Yixi Zhou, Fan Zhang, YU Chen, Haipeng Zhang, Preslav Nakov, and Zhuohan Xie. 2026. FinCARDS: Card-based analyst reranking for financial document question answering. In *Findings of ACL*.

Fengbin Zhu, Wenqiang Lei, Youcheng Huang, Chao Wang, Shuo Zhang, Jiancheng Lv, Fuli Feng, and Tat-Seng Chua. 2021. TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In *Proceedings of ACL-IJCNLP*.

Fengbin Zhu, Ziyang Liu, Fuli Feng, Chao Wang, Moxin Li, and Tat-Seng Chua. 2024. TAT-LLM: A specialized language model for discrete reasoning over tabular and textual data. *arXiv:2401.13223*.